МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность системы управления информацией

Выполнил

М. А. Хомич,

студент группы АС-68/24

Проверил

А. А. Крощенко,

доцент Кафедры ИИТ,

«__» _____2026 г.

Брест 2026

Цель: изучить применение линейной и логистической регрессии для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты.

Вариант 10.

Вариант 10

- **Регрессия (Прогнозирование счастья в странах)**
  1. World Happiness Report
  2. Предсказать оценку счастья (Score)
  3. **Задания:**
     - загрузите данные. В качестве признаков используйте GDP per capita, Social support, Healthy life expectancy;
     - обучите модель **линейной регрессии**;
     - рассчитайте **MSE** и **R^2**;
     - визуализируйте зависимость Score от GDP per capita с линией регрессии.
- **Классификация (Прогнозирование оттока клиентов)**

1. Telco Customer Churn
2. Предсказать, уйдёт ли клиент (Churn = 'Yes')
3. **Задания:**
   - загрузите данные, обработайте категориальные признаки;
   - обучите модель **логистической регрессии**;
   - рассчитайте **Accuracy**, **Precision** и **Recall** для класса 'Yes';
   - постройте **матрицу ошибок**.

Код программы:

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import (
    mean_squared_error,
    r2_score,
    accuracy_score,
    precision_score,
    recall_score,
    confusion_matrix,
    ConfusionMatrixDisplay
)
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
```

```
# Пути к файлам
HAPPINESS_PATH =
r"C:\Users\widuxd\PycharmProjects\ml_as68\reports\Khomich\2\src\world_happiness_report.csv"
CHURN_PATH = r"C:\Users\widuxd\PycharmProjects\ml_as68\reports\Khomich\2\src\Telco-Customer-Churn.csv"
REPORT_DIR = r"C:\Users\widuxd\PycharmProjects\ml_as68\reports\Khomich\2\rep"

os.makedirs(REPORT_DIR, exist_ok=True)

# 1. ЛИНЕЙНАЯ РЕГРЕССИЯ

print("\nЛИНЕЙНАЯ РЕГРЕССИЯ")

df = pd.read_csv(HAPPINESS_PATH)

features = [
    "GDP per capita",
    "Social support",
    "Healthy life expectancy"
]
target = "Score"

df = df[features + [target]].copy()

for column in features + [target]:
    df[column] = pd.to_numeric(df[column], errors="coerce")

df = df.dropna()

X = df[features]
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

regression_model = LinearRegression()
regression_model.fit(X_train, y_train)

y_pred = regression_model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Среднеквадратичная ошибка (MSE): {mse:.4f}")
print(f"Коэффициент детерминации (R²): {r2:.4f}")

print("\nКоэффициенты линейной регрессии:")

for feature, coefficient in zip(features, regression_model.coef_):
    print(f"{feature}: {coefficient:.4f}")

print(f"Свободный коэффициент: {regression_model.intercept_:.4f}")
```

```
# График зависимости Score от GDP per capita

plt.figure(figsize=(10, 6))

import seaborn as sns
sns.set_theme(style="whitegrid")

sns.regplot(
    data=df,
    x="GDP per capita",
    y="Score",
    scatter_kws={"alpha": 0.7, "color": "blue"},
    line_kws={"color": "red", "linewidth": 2}
)

plt.title(
    "Зависимость оценки счастья (Score) от ВВП на душу населения (GDP per capita)",
    fontsize=14
)
plt.xlabel("ВВП на душу населения", fontsize=12)
plt.ylabel("Score (Happiness)", fontsize=12)

plt.tight_layout()

plt.savefig(
    r"C:\Users\widuxd\PycharmProjects\ml_as68\reports\Khomich\2\rep\happiness_regression.png",
    dpi=300
)

plt.show()

# ============================================================
# 2. ЛОГИСТИЧЕСКАЯ РЕГРЕССИЯ — TELCO CUSTOMER CHURN
# ============================================================

print("\n========== ЛОГИСТИЧЕСКАЯ РЕГРЕССИЯ ==========")

df_churn = pd.read_csv(CHURN_PATH)

# Удаляем идентификатор клиента
df_churn = df_churn.drop(columns=["customerID"], errors="ignore")

# Преобразуем TotalCharges в числовой формат
if "TotalCharges" in df_churn.columns:
    df_churn["TotalCharges"] = pd.to_numeric(
        df_churn["TotalCharges"], errors="coerce"
    )

# Преобразуем целевую переменную
df_churn["Churn"] = df_churn["Churn"].map({
    "Yes": 1,
    "No": 0
})
```

```
df_churn = df_churn.dropna(subset=["Churn"])

X = df_churn.drop(columns=["Churn"])
y = df_churn["Churn"].astype(int)

# Определяем типы признаков
numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()

categorical_features = X.select_dtypes(
    include=["object", "category", "bool"]
).columns.tolist()

# Подготовка числовых признаков
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Подготовка категориальных признаков
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

# Объединение обработки признаков
preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

# Создание модели логистической регрессии
churn_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=2000))
])

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

churn_model.fit(X_train, y_train)

y_pred = churn_model.predict(X_test)



accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision для класса Yes: {precision:.4f}")
print(f"Recall для класса Yes: {recall:.4f}")


# Матрица ошибок

cm = confusion_matrix(y_test, y_pred, labels=[0, 1])

display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No", "Yes"]
)

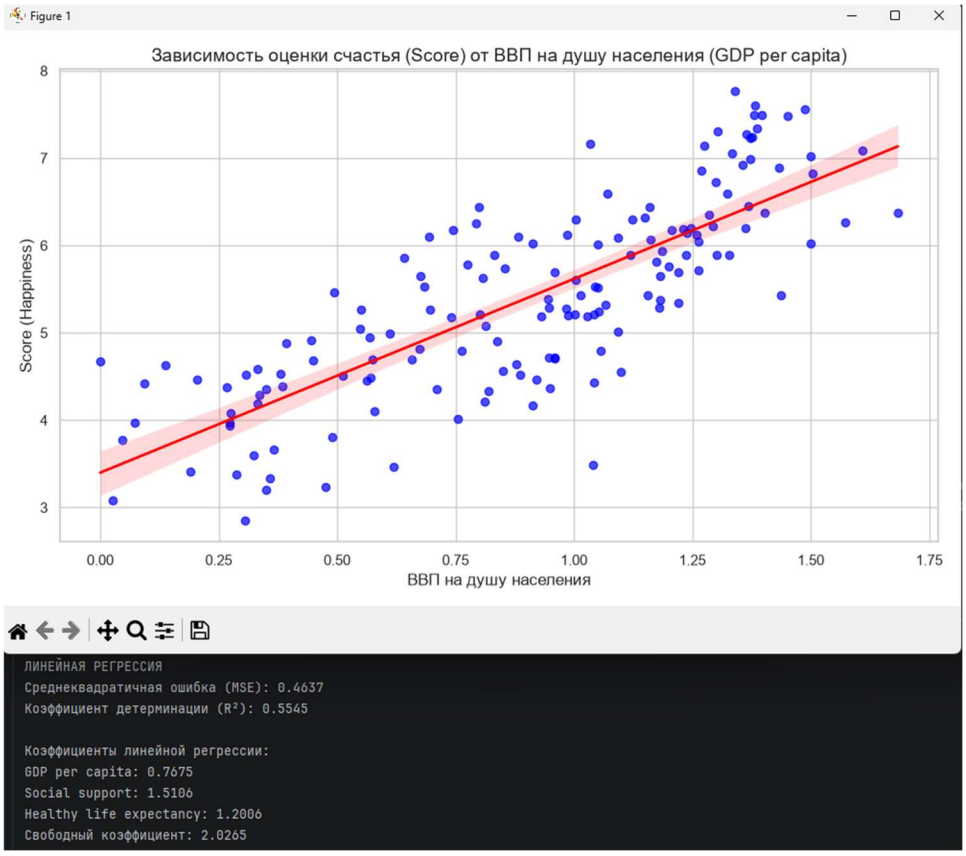
display.plot(cmap="Blues", values_format="d")
plt.title("Матрица ошибок — Telco Customer Churn")
plt.tight_layout()

plt.savefig(
    r"C:\Users\widuxd\PycharmProjects\ml_as68\reports\Khomich\2\rep\churn_confusion_matrix.png",
    dpi=300
)

plt.show()

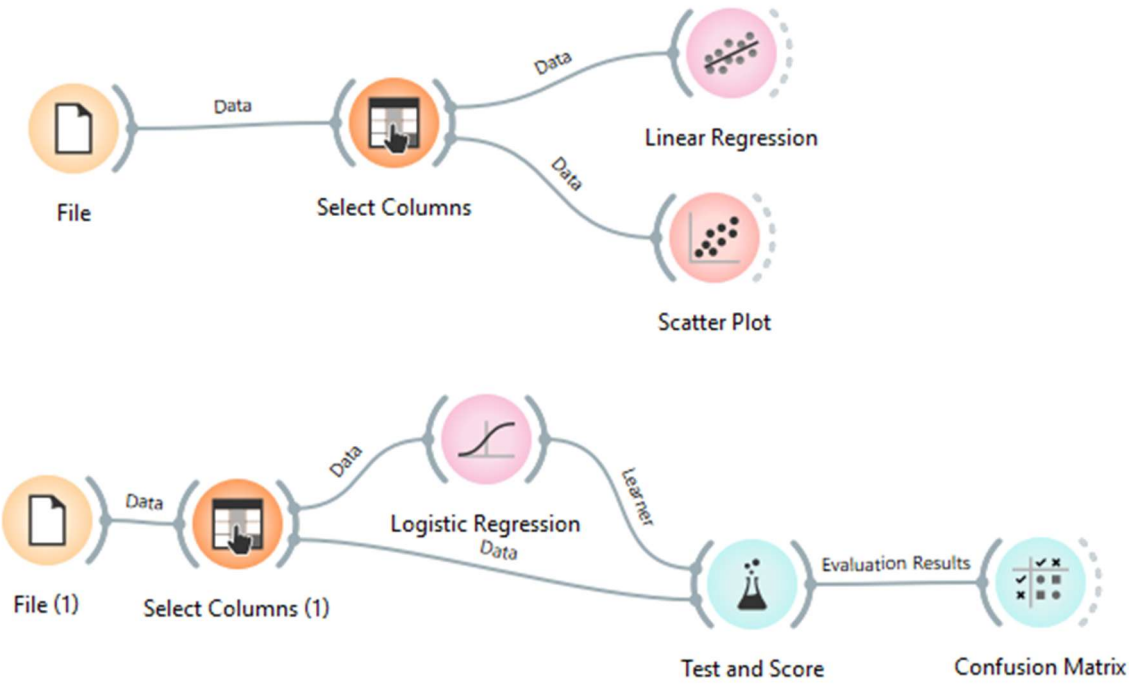
print("\nРабота программы завершена.")
```

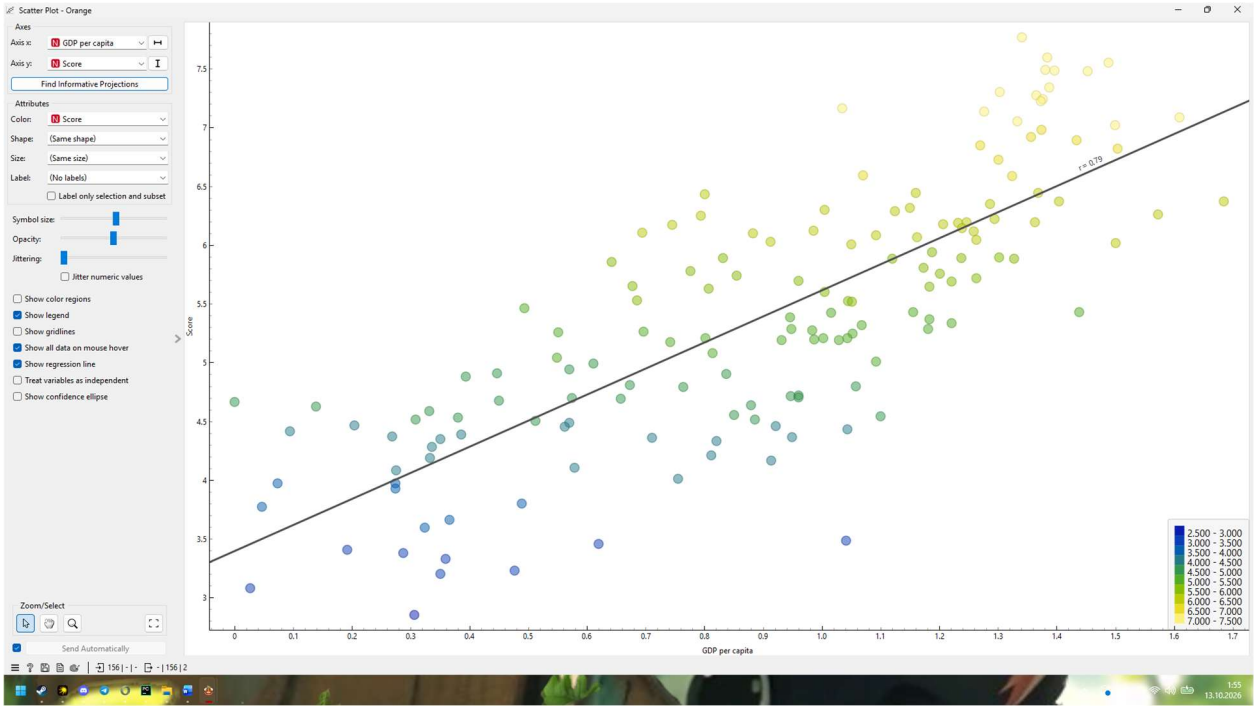
Результаты работы программы:

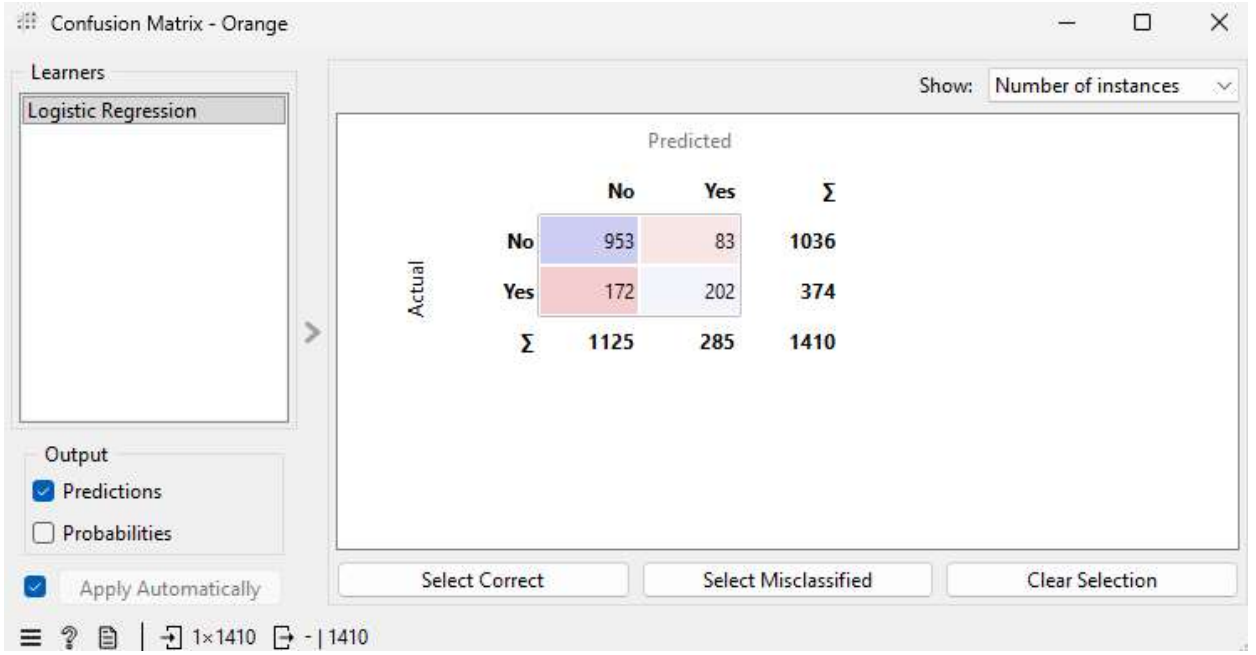




Результаты моделирования в Orange:







Вывод: в ходе лабораторной работы были изучены основы машинного обучения, построены модели регрессии и классификации в Python и Orange Data Mining, а также выполнена оценка их качества. Полученные результаты позволили закрепить навыки подготовки данных и применения методов машинного обучения.